PREDICCIÓN DEL ÉXITO COMERCIAL DE VIDEOJUEGOS MEDIANTE INTELIGENCIA ARTIFICIAL

Ainhoa Lodeiro López

Curso de especialización de IA y Big Data

ÍNDICE

- Planteamiento del problema
- Obtención y tratamiento de datos
- Muestra de datos
- Desarrollo de la solución IA/Big Data
  - Random Forest Regressor
  - Redes Neuronales
  - Comparativa
- Conclusión

PLANTEAMIENTO DEL PROBLEMA

- Desarrollar un videojuego implica grandes inversiones. Evaluar su potencial comercial antes del lanzamiento es clave para tomar decisiones estratégicas y minimizar riesgos: marketing, motor gráfico, localización, jugabilidad.
- El **mercado europeo**, diverso y estratégico, requiere análisis específicos para maximizar la rentabilidad.
- Se propone una **herramienta basada en IA** que analice datos de videojuegos anteriores (género, plataforma, editor, etc.).
- Objetivo: **apoyar a estudios de desarrollo** (especialmente pequeños) a tomar decisiones informadas, reducir riesgos y optimizar recursos.

	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.01	3.28	2.96	33.00
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	1.00	31.37

```
videojuegos.shape
```

```
(16598, 11)
```

```
[156] videojuegos.dropna(inplace = True)
```

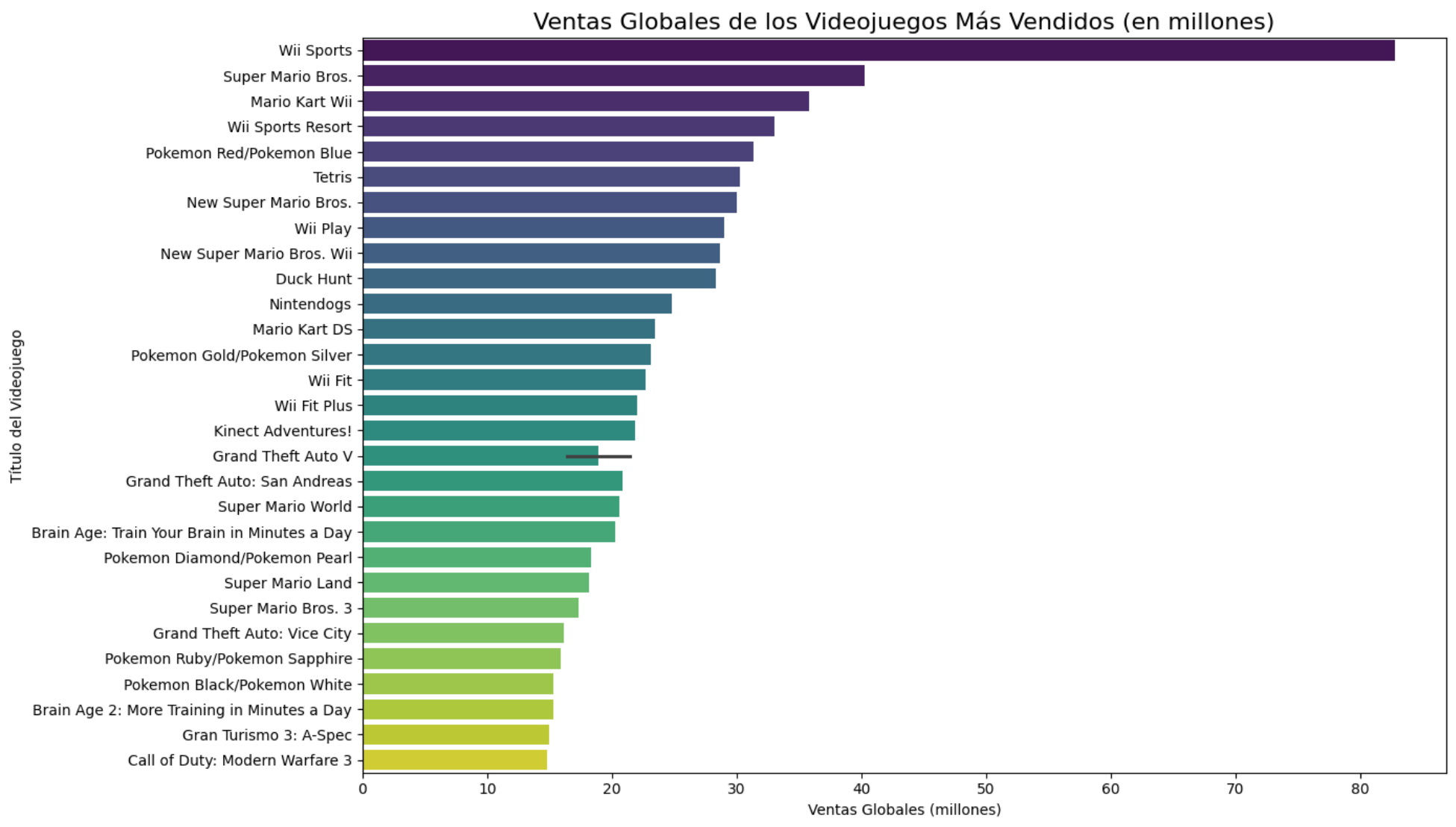
```
videojuegos.isnull().sum()
```

	0
Rank	0
Name	0
Platform	0
Year	271
Genre	0
Publisher	58
NA_Sales	0
EU_Sales	0
JP_Sales	0
Other_Sales	0
Global_Sales	0

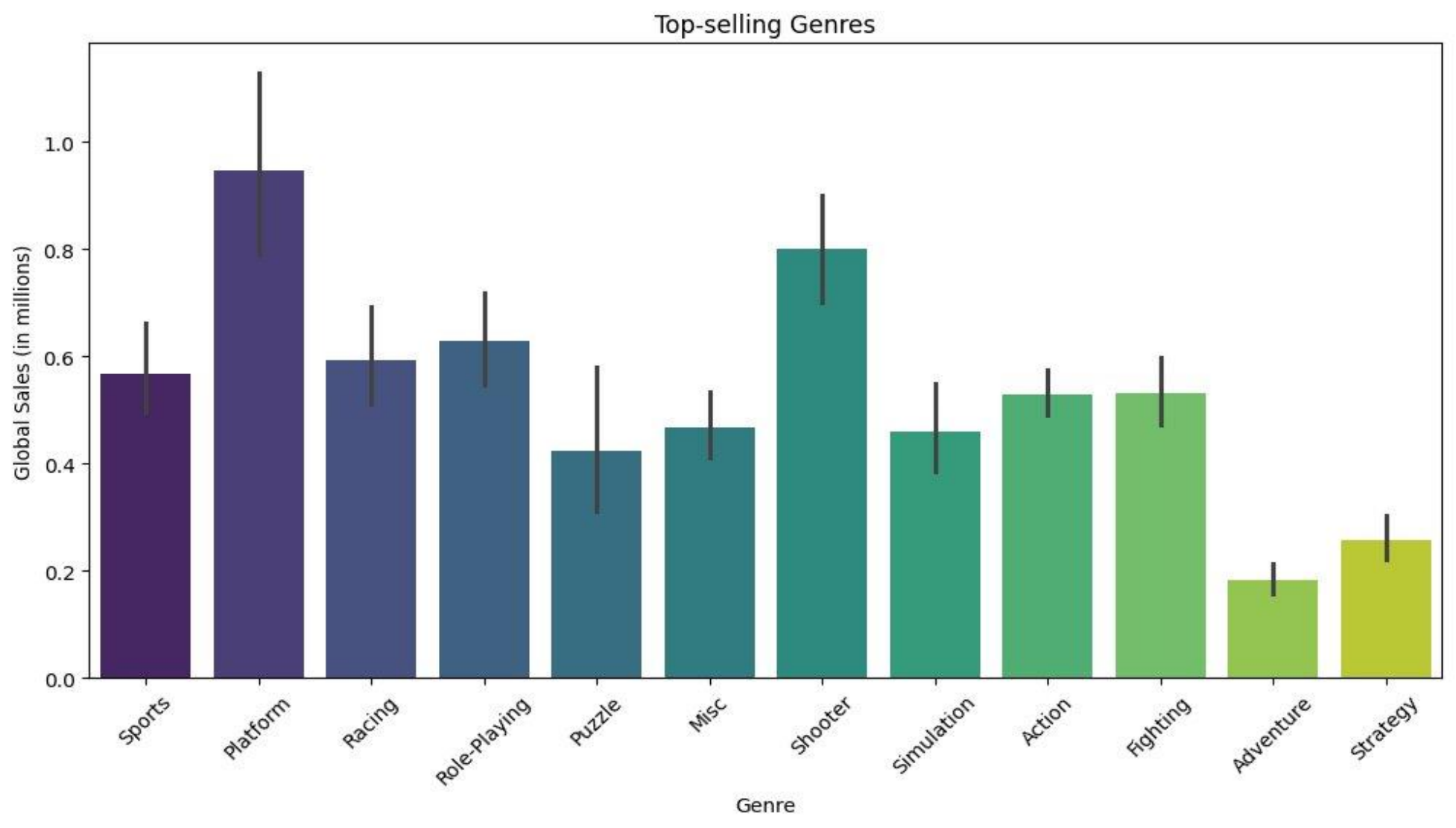
dtype: int64

OBTENCIÓN Y TRATAMIENTO DE DATOS

MUESTRA DE DATOS



MUESTRA DE DATOS



DESARROLLO DE SOLUCIÓN IA/BIG DATA: RANDOM FOREST REGRESSOR

- Numpy: Fundamental para trabajar con vectores, matrices y operaciones numéricas.
- Pandas: Permite la carga, explora y la manipulación de los datos fácilmente
- Sklearn: Libreria de machine learning muy completa, con las siguientes funciones:
  - Train_test: Divide los datos en un conjunto de entrenamiento y prueba
  - Mean_squared_error: Promedio del error cuadrado entre lo real y lo predicho
  - R2_score: Mide qué tan bien el modelo explica la variación en los datos
- LabelEncoder: Convierte columnas de texto en números
- RandomForestRegressor: Modelo de aprendizaje automático basado en muchos árboles de decisión (problemas de regresión)

DESARROLLO DE SOLUCIÓN IA/BIG DATA: RANDOM FOREST REGRESSOR

```
le_platform = LabelEncoder()
le_genre = LabelEncoder()
le_publisher = LabelEncoder()

videojuegos['Platform'] = le_platform.fit_transform(videojuegos['Platform'])
videojuegos['Genre'] = le_genre.fit_transform(videojuegos['Genre'])
videojuegos['Publisher'] = le_publisher.fit_transform(videojuegos['Publisher'])
```

```
[160] X = videojuegos[['Platform', 'Year', 'Genre', 'Publisher', 'NA_Sales', 'JP_Sales', 'Other_Sales']]
      y = videojuegos['EU_Sales']
```

```
[12] X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
[13] model = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
model.fit(X_train, y_train)
```

```
[15] y_pred = model.predict(X_test)
```

DESARROLLO DE SOLUCIÓN IA/BIG DATA: RANDOM FOREST REGRESSOR

```
[16] mse = mean_squared_error(y_test, y_pred)
     r2 = r2_score(y_test, y_pred)

     # Mostrar los resultados
     print(f"Error cuadrático medio (MSE): {mse}")
     print(f"Coeficiente de determinación (R²): {r2}")
```

```
Error cuadrático medio (MSE): 0.1495491278272172
Coeficiente de determinación (R²): 0.7085667398024575
```

```
nuevo_juego = {
    'Platform': 'PC',
    'Year': 2025,
    'Genre': 'Action',
    'Publisher': 'Capcom',
    'NA_Sales': 10.0,
    'JP_Sales': 2.5,
    'Other_Sales': 5.3
}

nuevo_df = pd.DataFrame([nuevo_juego])

nuevo_df['Platform'] = le_platform.transform(nuevo_df['Platform'])
nuevo_df['Genre'] = le_genre.transform(nuevo_df['Genre'])
nuevo_df['Publisher'] = le_publisher.transform(nuevo_df['Publisher'])

prediccion = model.predict(nuevo_df)
print(f"Predicción de ventas en Europa: {prediccion[0]:.2f} millones")
```

```
Predicción de ventas en Europa: 8.89 millones
```

Visualización del modelo por StreamLit:

- [Proyecto Random Forest Regressor]

DESARROLLO DE SOLUCIÓN IA/BIG DATA: REDES NEURONALES

```
[60] import tensorflow as tf
     from tensorflow import keras
```

TensorFlow: Proporciona las herramientas necesarias para construir, entrenar y evaluar redes neuronales de manera eficiente y flexible.

```
[55] videojuegos = pd.read_csv("Video_Games_Sales.csv")
     videojuegos.dropna(inplace=True)
```

```
[56] le_platform = LabelEncoder()
     le_genre = LabelEncoder()
     le_publisher = LabelEncoder()

     videojuegos['Platform'] = le_platform.fit_transform(videojuegos['Platform'])
     videojuegos['Genre'] = le_genre.fit_transform(videojuegos['Genre'])
     videojuegos['Publisher'] = le_publisher.fit_transform(videojuegos['Publisher'])
```

```
[57] X = videojuegos[['Platform', 'Year', 'Genre', 'Publisher', 'NA_Sales', 'JP_Sales', 'Other_Sales']]
     y = videojuegos['EU_Sales']
```

```
[58] scaler = StandardScaler()
     X_scaled = scaler.fit_transform(X)
```

```
[59] X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

DESARROLLO DE SOLUCIÓN IA/BIG DATA: REDES NEURONALES

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)
])
```

```
[ ] model.compile(optimizer='adam', loss='mse', metrics=['mae'])

history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2, verbose=1)

Epoch 1/20
326/326 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.1666 - mae: 0.1683 - val_loss: 0.0856 - val_mae: 0.0875
```

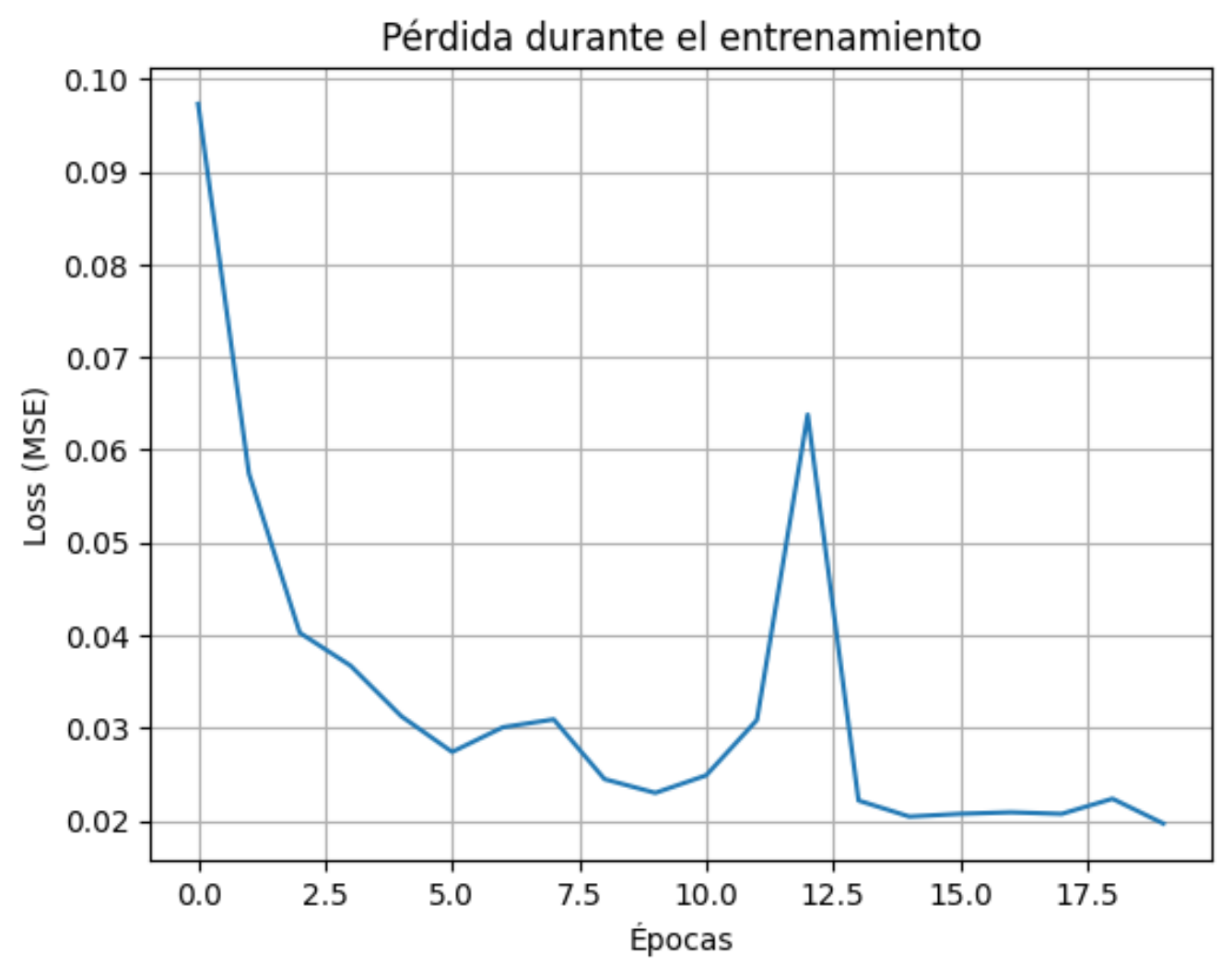
```
y_pred_nn = model.predict(X_test).flatten()

mse_nn = mean_squared_error(y_test, y_pred_nn)
r2_nn = r2_score(y_test, y_pred_nn)

print(f"Error cuadrático medio (MSE) - Red Neuronal: {mse_nn}")
print(f"Coeficiente de determinación (R²) - Red Neuronal: {r2_nn}")

102/102 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
Error cuadrático medio (MSE) - Red Neuronal: 0.051583453760352976
Coeficiente de determinación (R²) - Red Neuronal: 0.8994769523564357
```

DESARROLLO DE SOLUCIÓN IA/BIG DATA: REDES NEURONALES



DESARROLLO DE SOLUCIÓN IA/BIG DATA: REDES NEURONALES

```python
nuevo_juego = {
    'Platform': 'PC',
    'Year': 2025,
    'Genre': 'Action',
    'Publisher': 'Capcom',
    'NA_Sales': 15.0,
    'JP_Sales': 2.5,
    'Other_Sales': 10.3
}

nuevo_df = pd.DataFrame([nuevo_juego])
nuevo_df['Platform'] = le_platform.transform(nuevo_df['Platform'])
nuevo_df['Genre'] = le_genre.transform(nuevo_df['Genre'])
nuevo_df['Publisher'] = le_publisher.transform(nuevo_df['Publisher'])

nuevo_df_scaled = scaler.transform(nuevo_df)

prediccion_nn = model.predict(nuevo_df_scaled)
print(f"Predicción de ventas en Europa (red neuronal): {prediccion_nn[0][0]:.2f} millones")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
Predicción de ventas en Europa (red neuronal): 18.29 millones
```

Visualización del modelo por StreamLit:

- Proyecto Redes Neuronales

COMPARATIVA

MÉTRICA	RANDOM FOREST	RED NEURONAL
MSE	0.15	0.015
R2	0.70	0.89

Habiendo realizado dos modelos, toca el momento de comparar los resultados y entender porque uno es mejor que el otro. Primero interpretando las métricas, recordando que el MSE (Mean Squared Error) mide el promedio del cuadrado de los errores entre las predicciones del modelo y los valores reales, siendo que si da un valor bajo indica buena precisión, mientras que el R2 (R-squared) indica que también se ajusta el modelo a los datos reales, sabiendo que cuanto más cerca de 1 mejor.

Se puede observar bien que la red neuronal supera al Random Forest en ambas métricas. Por un lado, el MSE es diez veces menor lo que indica que las predicciones son mucho más cercanas a los valores reales. Por otro lado, el R2 de la Red neuronal explica que el 89% de la varianza en las ventas europeas, por lo tanto, se ve mucho mejor que el 70% de la varianza del otro modelo.

Por la capacidad que tiene para modelar relaciones no lineales complejas, ya que la relación entre las ventas de distintas regiones y las características del juego es no lineal y multifactorial, lo que indica que hay muchos valores que influyen en el resultado .
Tiene un mejor ajuste al problema específico, en cambio Random Forest puede no captar todas las complejidades de interacciones entre las variables.

CONCLUSIÓN

- **La red neuronal** fue el modelo más preciso para estimar ventas en Europa (menor MSE y mayor R^2).
- **La calidad y cantidad de datos** influyen fuertemente en el rendimiento del modelo.
- **Variables ausentes importantes**:
  - Puntuaciones de críticos y usuarios
  - Competencia en el mercado (actual y futura)
  - Precio del juego
  - Tendencias culturales y regionales
- **Limitación clave**: Muchos factores clave no son cuantificables fácilmente o no están disponibles en el dataset.
- **Conclusión**: El modelo ofrece una **buena estimación inicial**, pero para decisiones estratégicas es necesario un análisis más profundo y actualizado del mercado.